Temat projektu: Przydział pokoi simulated annealing metodą Monte Carlo
Autor: Paweł Lejczak, Szymon Gaweł
Data: 28 kwietnia 2026

1. Wstęp

W zadaniu rozpatrujemy NP-trudny problem przydziału pokoi. Dla każdego gościa *i* oraz każdego pokoju *j* istnieje liczba całkowita *pref[i][j]* ∈ *[0, MAX_PREF]* określająca preferencję gościa względem danego pokoju (im wyższa wartość, tym lepiej). Szukamy takiej bijekcji π, że całkowity koszt jest minimalny (poniższy wzór). Dla N = 20 liczba możliwych permutacji wynosi 20! co uniemożliwia podejście typu “brute force, dlatego nasz projektu implementuje metaheurystykę - metoda symulowanego wyżarzania, zrównoleglona z użyciem MPI.

$$\mathrm{cost}(\pi) = \sum_{i=0}^{N-1} \big(\mathrm{MAX_PREF} - \mathrm{pref}[i][\pi(i)]\big)$$

2. Symulowane wyżarzanie

Czyli probabilistyczna technika optymalizacji globalnej, inspirowaną procesem kontrolowanego studzenia metali (wyżarzanie). Zgodnie z założeniami projektu zastosowano krok Monte Carlo, który wyraża się losową modyfikacją stanu (zamiana pokoi dwóch losowo wybranych gości). Wyjaśniając analogię metalurgiczną, stan metalu to permutacja, jego energia to bieżąca funkcja kosztu, a temperatura jest parametrem kontrolującym prawdopodobieństwo akceptacji gorszych rozwiązań (temperatura stopniowo się obniża, gdzie niska temperatura końcowa oznacza dominację najlepszych rozwiązań w naszym algorytmie).

3. Model równoległy - MPI

Biblioteka MPI implementuje tzw. model wyspowy z wymianą elit, co zdecydowanie przyspiesza obliczania i poprawia jakość rozwiązań względem klasycznego podejścia sekwencyjnego. “Flow” działania programu dla wspomnianego podejścia jest następujący:

- Każdy P procesów MPI uruchamia niezależny łańcuch SA z innym ziarnem losowym.
- Co SYNC_INTERVAL iteracji (np. 10 000) procesy komunikują się i obliczają globalnie najlepszy koszt i numer procesu, który go osiągnął (operacja MPI_Allreduce z MPI_MINLOC), najlepsze rozwiązanie rozsyłane jest do wszystkich procesów - MPI_Bcast.

- Dobre rozwiązania rozprzestrzeniają się między wyspami, zapobiegając przedwczesnej zbieżności do słabego lokalnego minimum, dodatkowo komunikacja odbywa się tylko co SYNC_INTERVAL, więc narzut jest niewielki w porównaniu z liczbą iteracji.

4. Impelmentacja

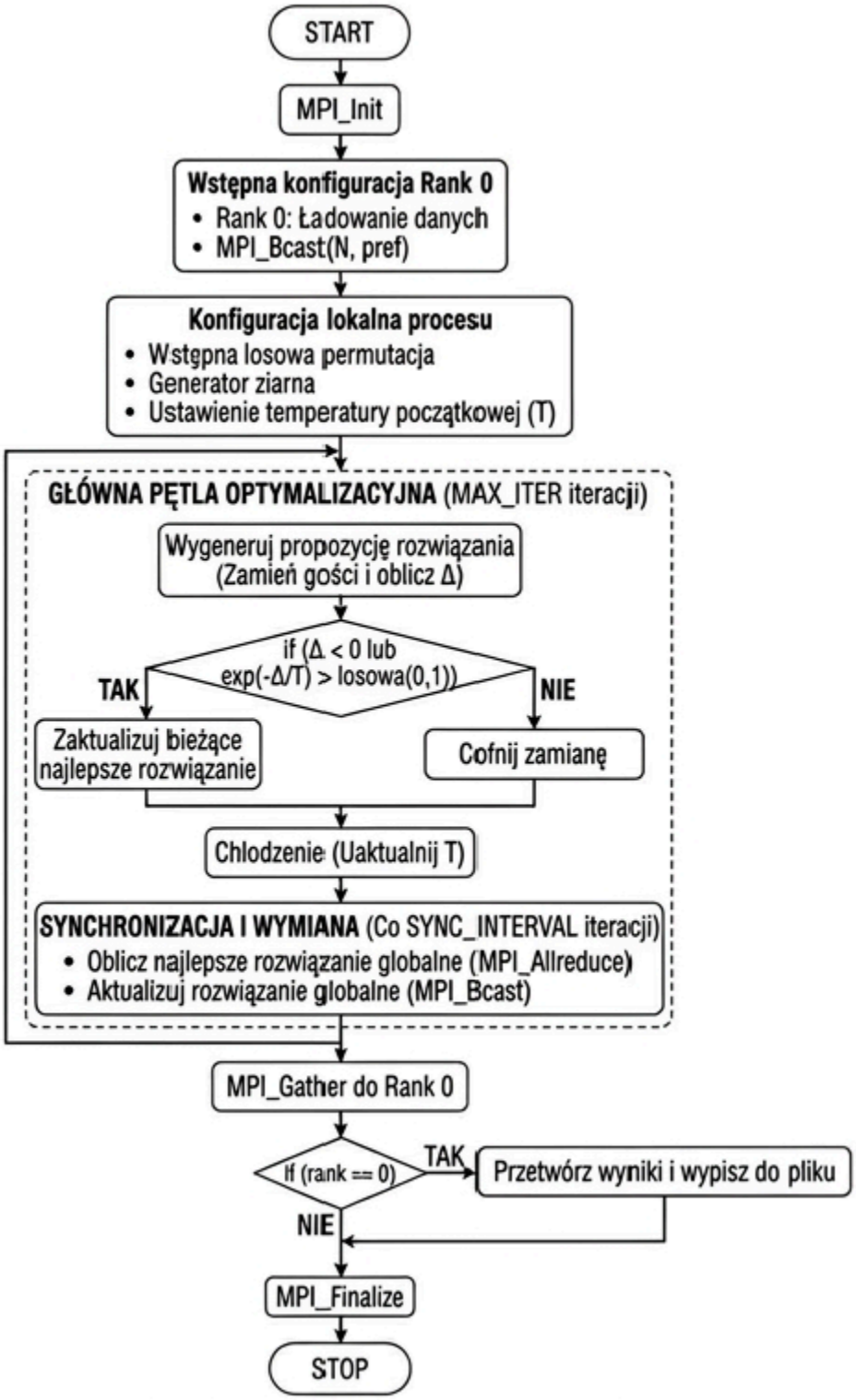
- Macierz preferencji pref[MAX_N][MAX_N] przechowuje dane dla gości, permutacje (przydziały) reprezentowane są jako tablice długości N, gdzie current[i] to numer pokoju przydzielonego gościowi i.

- Proces 0 odczytuje z plik wejściowy- liczbę N oraz macierz preferencji, a następnie rozsyła je do wszystkich procesów za pomocą MPI_Bcast. Każdy proces alokuje pamięć dla swoich stanów: current, best_local (najlepsze rozwiązanie lokalne) i gbest (bufor na rozwiązanie globalne). Stan początkowy to losowa permutacja.

- Symulowanie możemy rozumieć jako ciąg iteracji. W każdej iteracji wybieramy dwóch różnych gości g1, g2 i próbujemy zamienić ich pokój, jeśli zmiana funkcji kosztu jest korzystna zmianę zostawiamy, jeśli nie to cofamy. Co 10 000 iteracji procesy uruchomione na różnych maszynach wymieniają się informacjami.

- Po zakończeniu wszystkich iteracji każdy proces wysyła do procesu 0 swój najlepszy koszt i permutację. Proces 0 wybiera rozwiązanie o minimalnym koszcie i wypisuje ja do pliku wyjściowego.

5. Uproszczony schemat działania programu



6. Analiza wyników

- Dla $N = 20$ i 3 procesów MPI uzyskano:
- Koszt globalny wyniósł 3
- Maksymalny możliwy koszt to 200
- Średnia preferencja to $(200 - 3 = 197) / 20$, czyli $9.85/10$
- Czas naszego wykonania jest bardzo krótki, wyniósł 0.06 s - zastosowano podejście programowania równoległego, sens zastosowania tego podejścia rośnie wraz z zwiększającą się liczbą gości. W weryfikacji program sprawdził, że otrzymana permutacja jest poprawna, czyli że każdy pokój użyto dokładnie raz.

7. Wnioski

- Każdy proces MPI uruchamia niezależny łańcuch symulowanego wyżarzania (inny stan początkowy, inne ziarno losowe). Dzięki temu eksplorujemy więcej rozwiązań w tym samym czasie rzeczywistym.

- Co $10\,000$ iteracji procesy wymieniają swoje najlepsze rozwiązania. Powoduje to rozprzestrzenianie się elit – dobre rozwiązania szybko trafiają do wszystkich wysp.

- Komunikacja odbywa się tylko ok. 100 razy podczas całego przebiegu ($\text{MAX_ITER} / \text{SYNC_INTERVAL} = 10^6 / 10^4 = 100$). Dlatego narzut sieciowy jest niewielki (poniżej 5% czasu obliczeń dla umiarkowanej liczby procesów).

- Zwiększenie liczby procesów nie skraca czasu wykonania (bo każdy wykonuje tyle samo iteracji), ale podnosi jakość znalezionego rozwiązania – rośnie różnorodność ścieżek przeszukiwania. Można powiedzieć, że skraca czas znalezienia lepszego rozwiązania.

- Program weryfikuje poprawność permutacji po zebraniu wyników (proces 0 sprawdza, czy każdy pokój użyty dokładnie raz). Chroni to przed ewentualnymi błędami komunikacyjnymi lub uszkodzeniem pamięci.